

Game

Doors

- Currently your entity can't interact with doors
 - They block since **Move** return `failure()` ;
- Let's change that:
 - Add **OpenDoor** and **CloseDoor** actions
 - Edit **Move** to return an `alternative(OpenDoor)`

Detecting a door

- Doors and Tiles are separate objects
 - Think of a Tile as the ground/wall
 - Doors are on top of tiles
- Result: detect if tile has door, interact with Door object

Detecting a door

Once you have a position, you can get a Tile and test its type

```
Vec position{1, 1};
Tile& tile = engine.dungeon.get_tile(position);

// engine/dungeon/tile.h (simplified)
class Tile {
public:
    bool is_wall() const;
    bool has_door() const;
    bool walkable;
    std::shared_ptr<Door> door;
};
```

Retrieving a door object

Test if a tile has a door, then simply access the shared pointer on the tile:

```
class Tile {  
public:  
    bool has_door() const;  
    std::shared_ptr<Door> door;  
};  
...  
if (tile.has_door() && !tile.door->is_open()) {  
    return alternative(OpenDoor{*tile.door});  
}
```

Door class

```
// engine/dungeon/door.h
class Door {
public:
    Door(Tile& tile, bool is_horizontal, const Sprite& horizontal,
         const Sprite& vertical);
    bool is_open() const;
    void open();
    void close();
    const Sprite& get_sprite() const;
    const Tile& get_tile() const;

private:
    Tile& tile;
    bool open_status;
    bool is_horizontal;
    Sprite horizontal, vertical;
};
```

Door class

The most important member functions are:

```
// engine/dungeon/door.h
class Door {
public:
    bool is_open() const;
    void open();
    void close();
};
```

Opening a Door

- Access the Door object on Tile
- Call open ()
- Doors have a reference to their tile

```
Door::Door(Tile& tile, ...)
```

- So opening it does this:

```
void Door::open() {  
    open_status = true;  
    tile.walkable = true;  
}
```

Opening a Door



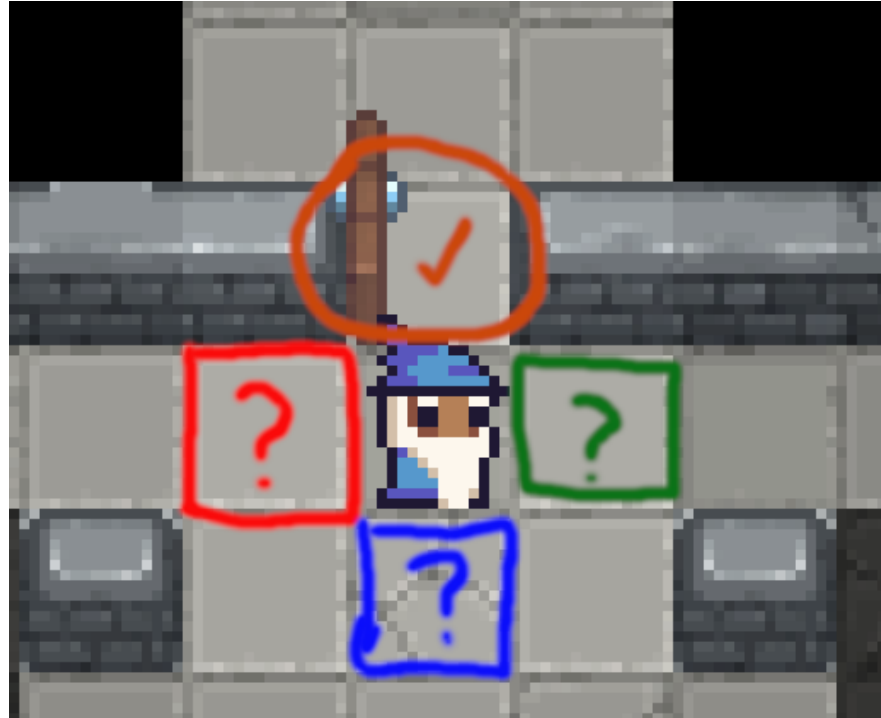
We know the door location given the movement
direction

Demo: Closing doors

To practice with Doors and learn more useful engine features, let's create a **CloseDoor** action!

Demo: Closing doors

Unlike in **OpenDoor** we don't know the direction



Search in surrounding tiles!

Demo: Closing doors

```
Vec pos{10, 10};  
std::vector<Vec> neighbors = engine.dungeon.neighbors(pos);  
for (Vec position : neighbors) {  
    Tile& tile = engine.dungeon.get_tile(position);  
    if (tile.has_door() && tile.door->is_open()) {  
        tile.door->close();  
    }  
}
```

Keybindings

- For closing doors, we should bind **CloseDoor** to a keypress.
- I used "C", you can choose whatever key you wish

Field of View issue



- I closed the door, but can still see behind it!
- Solution: Add an **Event** called **UpdateFOV**

Events

Events

Things that happen in the game

Action

What an **Entity** does for its turn

Basics

Two classes:

class Events

Manages events, you can

`Events::create_event<EventName>()`

class Event

Specifies what happened and for how long it will last

Event class

```
// engine/event.h
class Event {
public:
    Event(int number_of_frames=1); // how many frames it should last
    void update();

    // what the event does per update frame
    virtual void execute(Engine& engine) = 0;
    ...
};
```

Example Event

```
// content/events/updatefov.h
class UpdateFOV : public Event {
public:
    void execute(Engine& engine) override;
};

// content/events/updatefov.cpp
void UpdateFOV::execute(Engine& engine) {
    Vec hero_position = engine.hero->get_position();
    engine.dungeon.update_visibility(hero_position);
}
```

Why do you need Events?

If something happens in the game, we might **need to pause entities** taking turns

- Examples:
 - UpdateFOV: make sure the Hero can see newly visible tiles before next Entity moves
 - AttackAction → AnimationOfHit → HitEvent

UpdateFOV and CloseDoor

Update the field of view IF any doors where closed

```
if (closed_any_doors) { // boolean variable
    engine.events.create_event<UpdateFOV>();
    return success();
}
else {
    return failure();
}
```

UpdateFOV and OpenDoor

If you call `door.open()`, then update the Hero's field of view

```
engine.events.create_event<UpdateFOV>();
```

Monsters

We should add some Monsters to the game

```
int main() {
    try {
        Settings settings{"settings.txt"};
        Engine engine{settings};

        std::shared_ptr<Entity> hero = engine.create_hero();
        Heroes::make_knight(hero);

        for (int i = 0; i < 20; ++i) {
            std::shared_ptr<Entity> monster = engine.create_monster();
            Monsters::make_orc_masked(monster);
        }

        engine.run();
    }
    catch (std::exception& e) {
        std::cout << e.what() << '\n';
    }
}
```

Monsters

The process of adding monsters is similar to creating the hero:

```
std::shared_ptr<Entity> hero = engine.create_hero();  
Heroes::make_knight(hero); // customize  
  
std::shared_ptr<Entity> monster = engine.create_monster();  
Monsters::make_orc_masked(monster); // customize
```

Customizing monsters

```
// content/entities/monsters.cpp
void make_orc_masked(std::shared_ptr<Entity> monster) {
    monster->set_sprite("orc_masked");
    monster->set_max_health(12);
    monster->behavior = behavior;
}

std::unique_ptr<Action> behavior(Engine& engine, Entity& entity) {
    return std::make_unique<Rest>();
}
```

Monster names are listed in assets/monsters.txt

Behaviors

The default behavior is boring: Monsters just rest every turn

```
std::unique_ptr<Action> behavior(Engine& engine, Entity& entity)
    return std::make_unique<Rest>();
}
```

Let's make them more interesting

Updated behavior

```
// 66% of time Monster Wanders, otherwise Rests
std::unique_ptr<Action> behavior(Engine& engine, Entity& entity) {
    if (probability(66)) { // engine/util/randomness.h
        return std::make_unique<Wander>();
    }
    else {
        return std::make_unique<Rest>();
    }
}
```

Wander Action

```
// content/actions/wander.cpp
Result Wander::perform(Engine& engine) {
    Vec position = entity->get_position();
    std::vector<Vec> neighbors = engine.dungeon.neighbors(position);

    // randomize order of neighbors: shuffle in engine/utils/randomness.h
    shuffle(std::begin(neighbors), std::end(neighbors));

    // find first open floor tile
    for (const Vec& neighbor : neighbors) {
        Tile& tile = engine.dungeon.get_tile(neighbor);
        if (!tile.is_wall() && !tile.has_entity()) {
            Vec direction = neighbor - position;
            return alternative(Move{direction});
        }
    }

    return alternative(Rest{});
}
```

Even better

```
std::unique_ptr<Action> behavior(Engine& engine, Entity& entity) {  
    if (entity.is_visible() && engine.hero) {  
        Vec start = entity.get_position();  
        Vec goal = engine.hero->get_position();  
        std::vector<Vec> path = engine.dungeon.calculate_path(start, goal);  
  
        if (path.size() > 1) {  
            Vec direction = path.at(1) - path.at(0);  
            return std::make_unique<Move>(direction);  
        }  
    }  
  
    if (probability(66)) {  
        return std::make_unique<Wander>();  
    }  
    else {  
        return std::make_unique<Rest>();  
    }  
}
```

Creating Monsters in game

Engine has a member function:

```
class Engine {
public:
    // add a monster to the game
    std::shared_ptr<Entity> create_monster();
};

// in main.cpp
for (int i = 0; i < 20; ++i) {
    std::shared_ptr<Entity> monster = engine.create_monster();
    Monsters::make_orc_masked(monster);
}
```

Checkpoint 2

- Create `OpenDoor` Action (remember to add event `UpdateFOV`)
- Update `Move::perform` to return `alternative(OpenDoor{door object})` when appropriate
- Create two new Monsters (in addition to the one you made in class)
- Give all monsters a `behavior` function
- Add a few Monsters into your game (edit `main.cpp`)